

Anti-Interception Robust Trajectory Optimization for Unmanned Aerial Vehicles using Statistical Inference and Estimation

Vishwa Mehta

I. BACKGROUND

Implementation of algorithms for trajectory optimization of unmanned aerial vehicles (UAVs) is a challenging task due to the presence of uncertainties in the environment. The uncertainties can be due to the presence of wind, sensor noise, and other disturbances. In this project, we aim to develop a robust trajectory optimization algorithm for UAVs that can handle uncertainties in the environment. The algorithm will be based on statistical inference and estimation techniques. The algorithm mainly comprises of two parts: first, learning the trajectory of the interceptor (states and parameters) and using filtration and prediction techniques to estimate its future position. The second part is to optimize the trajectory of the UAV to avoid interception while accounting for the uncertainties in the environment. The algorithm will be tested on a simulated environment and the performance will be evaluated based on the interception rate and the computational complexity.

II. PROBLEM SETUP

The two parts described above are implemented as follows: The algorithm for state and parameter learning uses the noisy measurements of the interceptor's position (provided by the SLAM module, outside the scope of this project) to estimate the states and the parameters governing a simple kinematic model of the interceptor. The states of the interceptor are observed for a certain time period and the observation is terminated at a certain time to allow the UAV to optimize its trajectory based on the predictions made using an Extended Kalman Filter.

For the second part, this predicted state of the interceptor is used to optimize the trajectory of the UAV. The optimization problem is designed as an Optimal Control Problem using Multiple Shooting Method. A reference trajectory (which the UAV would have taken to reach the target in the absence of the interceptor) is used as the initial guess for the optimization problem and penalties on the deviations with respect to the states and controls at each time step are calculated to get the cost function for the optimization. The formulation for a Robust Trajectory Optimization is introduced by adding stochasticity to the inputs and states of the system at each time step. Using the Monte Carlo Algorithm, the likelihood of evasion is calculated for a given set of optimal inputs and is measured against a safety threshold. The optimization problem is solved iteratively until the safety threshold is met.

III. METHODOLOGY

A. UAV Trajectory Generation Algorithm

The aircraft dynamics are modeled using a 3-DOF model as shown in Figure 1. The model is based on the equations of motion for the UAV and the aerodynamic forces acting on it.

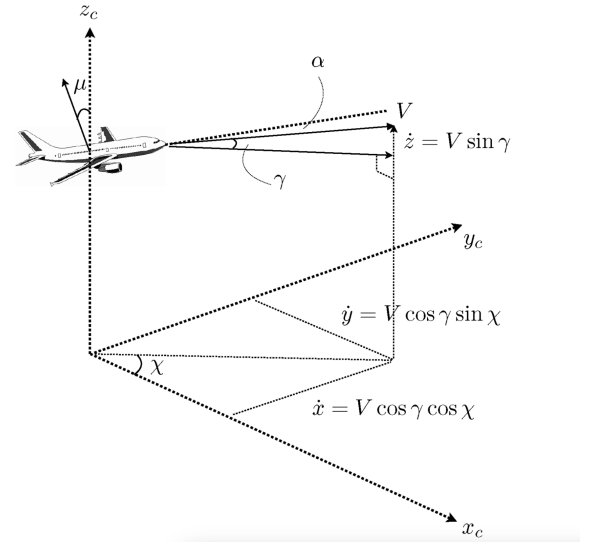


Fig. 1: 3-DOF Dynamics Model for the UAV

The states of the model are the horizontal position (x_c and y_c) and altitude z_c of the aircraft, the true airspeed V , the flight-path angle γ , and the heading angle χ . The control inputs to the model are the engine thrust T , the angle of attack α , and the roll angle μ . The state vector x is defined as $x \triangleq [x_c, y_c, z_c, \gamma, \chi, V]$ and the input vector u as $u \triangleq [\alpha, \mu, T]$.

The equations of motion are:

$$\dot{x}_c = V \cos \gamma \cos \chi \quad (1a)$$

$$\dot{y}_c = V \cos \gamma \sin \chi \quad (1b)$$

$$\dot{z}_c = V \sin \gamma \quad (1c)$$

$$\dot{\gamma} = \frac{1}{mV} [(L + T \sin \alpha) \cos \mu - mg \cos \gamma] \quad (1d)$$

$$\dot{\chi} = \frac{1}{mV \cos \gamma} [(L + T \sin \alpha) \sin \mu] \quad (1e)$$

$$\dot{V} = \frac{1}{m} [(T \cos \alpha - D) - mg \sin \gamma] \quad (1f)$$

The aerodynamic forces of lift and drag are denoted L and D , respectively, and the mass of the aircraft (assumed constant over the duration of the trajectory) by m . This model assumes that the aircraft can be controlled directly by changing the angle of attack and roll angles which is not true in practice and requires a cascaded control system. However, for the purpose of this project, we assume that the control inputs are directly related to the aircraft dynamics for the sake of simplicity. To keep the system realistic, upper and lower bounds are set on the control inputs and the states. The constraints are defined as:

$$-\pi/18 \leq \alpha \leq \pi/18 \quad (2a)$$

$$-\pi/18 \leq \mu \leq \pi/18 \quad (2b)$$

$$0 \leq T \leq 200 \quad (2c)$$

$$0 \leq V \leq 150 \quad (2d)$$

$$-\pi/6 \leq \gamma \leq \pi/6 \quad (2e)$$

$$-\pi/6 \leq \chi \leq \pi/6 \quad (2f)$$

B. Multiple Shooting Method and Optimal Control Problem

The dynamics of the UAV are propagated with respect to the control inputs using the Runge-Kutta 4th order method. The state at each time step is calculated using the following equation:

$$x_{k+1} = x_k + \Delta t \cdot f(x_k, u_k) \quad (3)$$

where $f(x_k, u_k)$ is the dynamics function of the UAV model and Δt is the time step. Instead of being solved as a single shooting problem, the optimal control problem is solved as a multiple shooting problem. The state at each time step is calculated using the dynamics function and the control inputs at that time step. The state at the end of the time horizon is calculated and the cost function is evaluated based on the deviation of the final state from the target state. The cost function for the problem is defined as

$$J_N(\mathbf{x}, \mathbf{u}) = \|\mathbf{x}_N - \mathbf{x}_N^r\|_P^2 + \sum_{i=0}^{N-1} (\|\mathbf{x}_i - \mathbf{x}_i^r\|_Q^2 + \|\mathbf{u}_i - \mathbf{u}_i^r\|_R^2) \quad (4)$$

where $\mathbf{P} \in \mathbb{R}^{6 \times 6}$, $\mathbf{Q} \in \mathbb{R}^{6 \times 6}$, and $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ are positive semidefinite, symmetric matrices adjusted to penalize the states or inputs as required. The calculation is initialized with a reference trajectory, $\{\mathbf{x}_i^r\}$ and $\{\mathbf{u}_i^r\}$. The reference trajectory is generated by time integration of the model from the initial state $\mathbf{x}_0$, with the inputs fixed as $\mathbf{u}(t_0)$ over the time horizon. Deviations from this trajectory on some or all of the states and inputs can then be penalized within Eq. (11).

C. Spatial Constraint for Evasion

The constraint uses the predicted threat location and blast radius provided by the Threat Estimation Module. The constraint is implemented based on the distance of the UAV at each state from the threat location. The constraint is defined as:

$$\|\mathbf{x}_i - \mathbf{x}_{\text{threat}}\|_2 \geq R_{\text{blast}} \quad (5)$$

where $\mathbf{x}_{\text{threat}}$ is the predicted threat location and R_{blast} is the radius of the threat. The constraint is added to the optimization problem and the problem is solved iteratively until the constraint is satisfied.

D. Reference Trajectory Generation for UAV

For this project, we assume the UAV to be a UCAV like the Shahed-136 Loitering Munition drone. Since loitering munition UAVs have to proceed with caution during their descent phase, we model the reference trajectory as a part of its descent phase. The reference trajectory is generated by integrating the dynamics of the UAV model with the control inputs fixed at the initial state. The reference trajectory is also modeled using a Trajectory Optimization problem with the objective being to minimize the altitude of the UAV and the range-to-go to the target. The reference trajectory is generated using the Multiple Shooting Method and the cost function is defined as:

$$J_N(\mathbf{x}, \mathbf{u}) = \left(\|\mathbf{x}(3)_{N-1}\|_2^2 + \|\mathbf{x}(1)_{\text{target}} - \mathbf{x}(1)_{N-1}\|_2^2 \right) \quad (6)$$

The reference trajectory is generated for a time horizon of 10 seconds and the states and inputs are stored at each time step. The reference trajectory is shown below:

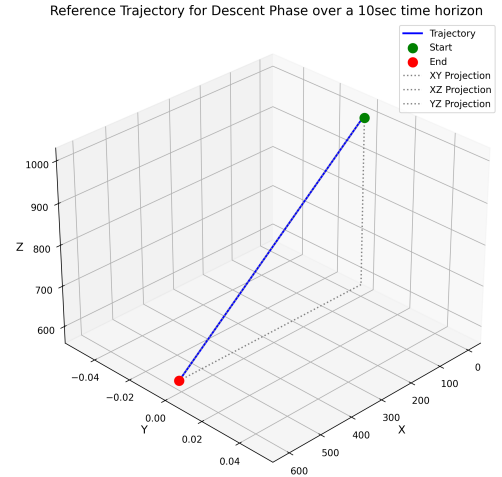


Fig. 2: Reference Trajectory for the UAV

E. State and Parameter Learning of the Interceptor for Open-Loop Prediction

1) *Interceptor Model:* To learn the states of an unknown interceptor, we assume the interceptor to be working on a simple kinematic model which can be generalized for any projectile moving at a fixed speed in any direction in the 3D space. A kinematic model is chosen in place of a dynamics model to keep the computation less expensive and to be able to generalize it for any given interceptor. For more robust algorithms, a dynamics model can be used to learn the states

of the interceptor. The chosen kinematic model is given by the following equations: Given the initial position (x_0, y_0, z_0) and constant velocity (V) and angles:

- θ_z : The angle in the vertical plane (pitch).
- θ_x : The angle in the horizontal plane (yaw).

The positions at any time t are computed as:

1) **Downrange position** $(x(t))$:

$$x(t) = x_0 + Vt \cos(\theta_z) \cos(\theta_x)$$

2) **Lateral position** $(y(t))$:

$$y(t) = y_0 + Vt \cos(\theta_z) \sin(\theta_x)$$

3) **Altitude** $(z(t))$:

$$z(t) = z_0 + Vt \sin(\theta_z)$$

Using these equations, we can write the propagation using Forward Euler Integration as:

$$\begin{bmatrix} x_{i+1} \\ y_{i+1} \\ z_{i+1} \end{bmatrix} = \begin{bmatrix} x_i + V \cos(\theta_z) \cos(\theta_x) \Delta t \\ y_i + V \cos(\theta_z) \sin(\theta_x) \Delta t \\ z_i + V \sin(\theta_z) \Delta t \end{bmatrix}$$

Now, since we are unaware of the initial position, the pitch and heading angles, and the velocity of the interceptor, we can learn these parameters to implement superior anti-interception algorithms that can make use of these parameters for live decision making. For the scope of this project, we learn these parameters along with the states in order to predict the future position of the interceptor. The states and parameters are learned using the Extended Kalman Filter (EKF) which is a recursive Bayesian filter that estimates the states of a dynamic system from a series of noisy measurements. The EKF is used to estimate the states and parameters of the interceptor using the noisy measurements of the interceptor's position. The states and parameters are learned for a certain time period and the observation is terminated at a certain time to allow the UAV to optimize its trajectory based on the predictions made using the EKF, hence utilizing the EKF for *Open-Loop Prediction* or *Forecasting* once the observation is terminated.

2) *Open-Loop Prediction or Forecasting*: Mathematically, we intend to find the Joint Posterior Distribution of the states and parameters of the interceptor given the measurements. The joint posterior distribution is given by:

$$P(X_N, \theta | \mathcal{Y}_n)$$

where θ is the parameter vector, X_N is the state vector at time N , $N > n$ (since the data observation is terminated early on for Trajectory Optimization) and $\mathcal{Y}_n$ is the set of measurements up to time n .

Using the law of total probability, the joint posterior can be written as:

$$P(X_N, \theta | \mathcal{Y}_n) = \int P(X_N | X_n, \theta) P(X_n, \theta | \mathcal{Y}_n) dX_n.$$

where:

- $P(X_n, \theta | \mathcal{Y}_n)$: The posterior distribution of the state X_n and parameters θ given observations $\mathcal{Y}_n$. This is obtained from a filtering process (e.g., EKF, particle filter).
- $P(X_N | X_n, \theta)$: The transition probability that propagates the state X_n to X_N based on the system dynamics.

The transition probability $P(X_N | X_n, \theta)$ accounts for the dynamics of the system and can be recursively defined as:

$$P(X_N | X_n, \theta) = \int P(X_N | X_{N-1}, \theta) P(X_{N-1} | X_n, \theta) dX_{N-1}.$$

Here:

- $P(X_k | X_{k-1}, \theta)$: Describes the state transition model (e.g., $X_k = \phi(X_{k-1}; \theta) + \xi$) where ξ is the process noise.

Combining the above steps, the full joint posterior for predicting X_N and parameters θ is:

$$P(X_N, \theta | \mathcal{Y}_n) = \int P(X_N | X_n, \theta) P(X_n, \theta | \mathcal{Y}_n) dX_n.$$

This equation describes the propagation of the posterior from time n (current time with data) to N (future time without data).

- $P(X_n, \theta | \mathcal{Y}_n)$: Filtered posterior distribution of the state and parameters given data up to time n .
- $P(X_N | X_n, \theta)$: Predictive transition probability for propagating the state to future time N .
- $P(X_N, \theta | \mathcal{Y}_n)$: The desired joint posterior distribution for prediction.

3) *Parameter - State Augmentation*: The parameter-state augmentation is done to learn the parameters of the interceptor along with the states. We can approximate the parameter inference problem as a filtering problem by introducing random walk dynamics for the parameters, with a very small process noise, according to:

$$X_{k+1} = \phi(X_k; \theta) + \xi$$

$$\theta_{k+1} = \theta_k + \tau, \quad \tau \sim \mathcal{N}(0, \Sigma), \quad \Sigma \ll 1$$

$$Y_{k+1} = h(X_k) + \eta$$

where X_k is the state vector, θ_k is the parameter vector, Y_k is the measurement vector, ϕ is the state transition function, h is the measurement function, ξ is the process noise, and η is the measurement noise.

The parameter vector is augmented to the state vector and the dynamics of the system are modified to include the parameters. The augmented state vector is given by:

$$\mathbf{X} = \begin{bmatrix} x \\ y \\ z \\ V \\ \theta_x \\ \theta_z \end{bmatrix}.$$

The augmented propagation model is hence given by:

$$\mathbf{X}_{k+1} = \begin{bmatrix} x_k + \Delta t V_k \cos(\theta_z) \cos(\theta_x) \\ y_k + \Delta t V_k \cos(\theta_z) \sin(\theta_x) \\ z_k + \Delta t V_k \sin(\theta_z) \\ V_k \\ \theta_x \\ \theta_z \end{bmatrix} = \phi(\mathbf{X}_k; \theta)$$

For the filtering equations, a standard Gaussian noise is added to the state-parameter propagation model, where the process noise is given by:

$$\xi_k, \quad \xi_k \sim \mathcal{N}(\mathbf{0}, \Sigma),$$

where

$$\Sigma = \begin{bmatrix} \Sigma_1 & 0 \\ 0 & \Sigma_2 \end{bmatrix}.$$

where Σ_1 and Σ_2 are the process noise for the states and parameters, respectively and $\Sigma_2 \ll 1$

For this project, the measurement model is assumed to be the observed position of the interceptor. The measurement model is given by:

$$\mathbf{Y}_k = h(\mathbf{X}_k) = \begin{bmatrix} x_k \\ y_k \\ z_k \end{bmatrix}$$

A measurement noise is added to the measurement model to account for the noise in the measurements. The measurement noise is given by:

$$\eta_k, \quad \eta_k \sim \mathcal{N}(\mathbf{0}, \Gamma),$$

where Γ is the measurement noise covariance matrix.

4) *Extended Kalman Filter for the Parameter-State Augmentation*: The Extended Kalman Filter works by linearizing the system dynamics and the measurement model around the current estimate of the states and parameters to make the problem analytically tractable. The linearization is done using a Taylor Series Expansion around the current estimate of the states and parameters. The linearized system dynamics and measurement model are given by:

$$\mathbf{A}_k = \frac{\partial \phi(\mathbf{X}_k)}{\partial \mathbf{X}_k}.$$

$$\mathbf{H}_k = \frac{\partial h(\mathbf{X}_k)}{\partial \mathbf{X}_k} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}.$$

The Prediction step mean and covariance are given by:

$$m_k^- \equiv \mathbb{E}[\mathbf{X}_k | \mathcal{Y}_{k-1}] = \phi(m_{k-1}, \theta_{k-1})$$

$$C_k^- = \text{Cov}[\mathbf{X}_k, \mathbf{X}_k | \mathcal{Y}_{k-1}] = \mathbf{A}_k C_{k-1} \mathbf{A}_k^T + \Sigma$$

The update step mean and covariance are given by:

$$m_k = m_k^- + U S^{-1} (y_k - \mu)$$

$$C_k = C_k^- - U S^{-1} U^T$$

where:

$$\mu \equiv \mathbb{E}[Y_k] = h(m_k^-)$$

$$U \equiv \text{Cov}[X_k, Y_k] = C_k^- \mathbf{H}_k^T$$

$$S \equiv \text{Cov}[Y_k, Y_k] = \mathbf{H}_k C_k^- \mathbf{H}_k^T + \Gamma$$

When the filtering phase is complete, the states and parameters are learned and the observation is terminated. The states and parameters are then used to predict the future position of the interceptor using the propagation model. The predicted position is then used to optimize the trajectory of the UAV to avoid interception.

5) *Simulating the Interceptor and Implementing the EKF for Filtering and Forecasting*: Based on the reference trajectory of the UAV, we simulate an Interception to be made at a certain time. The interceptor is assumed to be moving at a constant speed and the states and parameters are learned using the EKF. The simulated interception was such that the interceptor would reach the UAV at time $t = 5$ seconds. The data observation horizon is set to be till $t = 4$ seconds. The parameters used for the simulation are:

$$V = 224.5, \quad \theta_x = 0 \text{ rad}, \quad \theta_z = 0.83 \text{ rad}$$

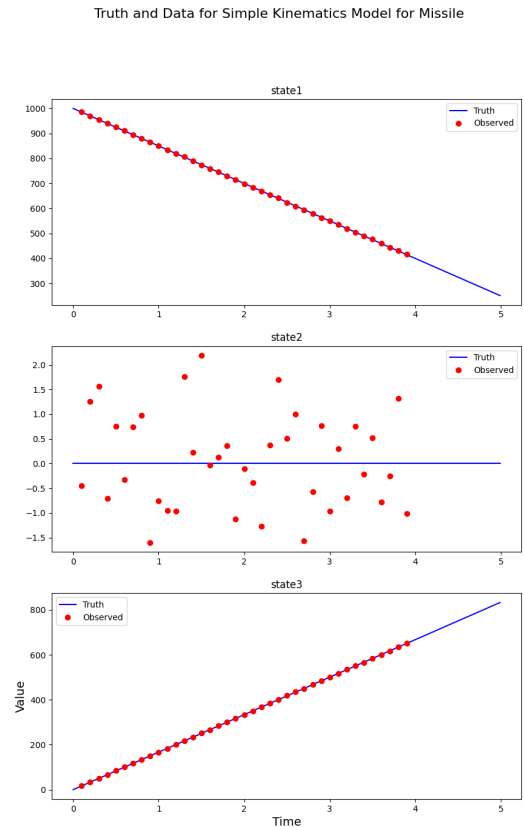


Fig. 3: Simulated data and truth for the Missile Trajectory

The states and parameters are learned using the EKF and the observation is terminated at $t = 4$ seconds. The states and parameters are then used to predict the future position of the interceptor at $t = 5$ seconds. The following figure shows

the simulated trajectory of the interceptor and the predicted position of the interceptor at $t = 5$ seconds along with the learned states and parameters at $t = 5$ seconds:

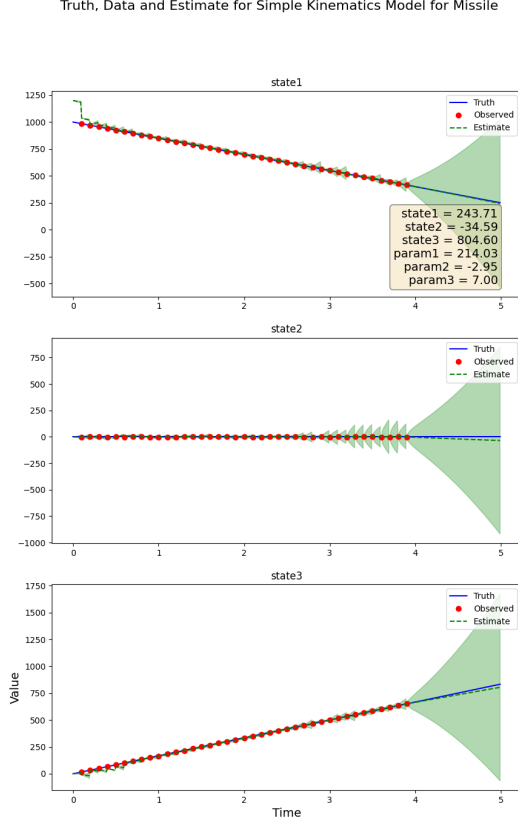


Fig. 4: Filtering and Forecasting for the Missile Trajectory

At $t = 5$ seconds, the predicted states for the missile are:

$$x = 243.71m, \quad y = -34.59m, \quad z = 804.60m$$

$$V = 214.03, \quad \theta_x = -2.95rad, \quad \theta_z = 7.00rad$$

We observe that the heading speed is found close to the actual value. However, the yaw and pitch angles are completely inaccurate. This might be an implication of the linearization done by the EKF over the functions of angles, leading to a poor estimation of the angles. This can be improved by using a more sophisticated non-linear filter like the Particle Filter.

F. Robust Trajectory Optimization for UAV

1) *Deterministic Optimal Control Problem:* The optimization algorithm for a deterministic anti-interception system is defined as:

$$\begin{aligned} & \min_{\mathbf{x}, \mathbf{u}} J_N(\mathbf{x}, \mathbf{u}) \\ & \text{subject to: (1), (2), (3), (4)} \\ & \text{using } \mathbf{u} \in \mathcal{U}, \end{aligned}$$

where the set $\mathcal{U}$ is the set of feasible control inputs. The optimization problem is solved using the Multiple Shooting Method and the optimal trajectory is obtained. The optimal trajectory is shown below:

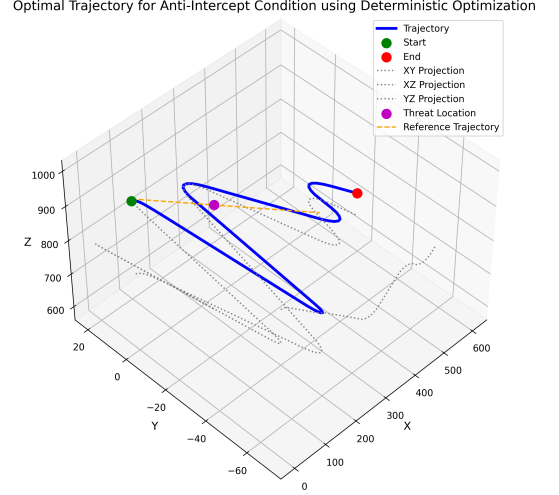


Fig. 5: Optimal Trajectory for the UAV using Deterministic Optimal Control

2) *Stochastic Optimal Control Problem using a Robustness Constraint:* Rather than utilizing the deterministic optimal control problem, we now introduce stochasticity to the inputs and states of the system at each time step. The stochasticity is introduced by adding Gaussian noise to the states and inputs at each time step. The noise is added to the states (in Eq. 3) and inputs as:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t \cdot f(\mathbf{x}_k, \mathbf{u}_k) + \epsilon_K$$

$$\mathbf{u}_{k+1} = \mathbf{u}_k + \delta_k$$

A robustness constraint is added to the optimization problem to ensure that the UAV has a high likelihood of evasion. The robustness constraint uses the current set of optimal outputs in the optimization problem, generates N particles at the initial state, and propagates through the system using the noisy inputs and states. An indicator function is used to determine if the particles come within the blast radius of the threat. The indicator function is given by:

$$1_{\text{success}}(\text{particle}_j) = \begin{cases} 1, & \text{if } \|\mathbf{x}_i - \mathbf{x}_{\text{threat}}\|_2 \geq R_{\text{blast}} \\ 0, & \text{otherwise} \end{cases}$$

The constraint is added to the optimization problem as:

$$g_{\text{robust}}(\mathbf{x}, \mathbf{u}) = \frac{1}{N} \sum_{j=1}^N 1_{\text{success}}(\text{particle}_j) \geq \text{threshold}$$

This constraint enables us to account for the uncertainties present in the model, sensors and environment, and allows

the UAV to evade the interceptor with a high likelihood. The optimization problem is solved iteratively until the robustness constraint is met. When applied to the earlier deterministic optimal controls, the optimal trajectory failed to meet the 0.99 threshold for the robustness constraint. This is due to the fact that the deterministic optimal control problem does not account for the uncertainties in the environment. The robustness constraint is hence necessary to ensure that the UAV has a high likelihood of evasion. Using the robust trajectory optimization algorithm, the optimal trajectory is obtained as shown below:

interception. The results show that the algorithm is successful in ensuring that the UAV can evade the interceptor with a high likelihood by using Stochasticity as a means to account for the uncertainties in the environment.

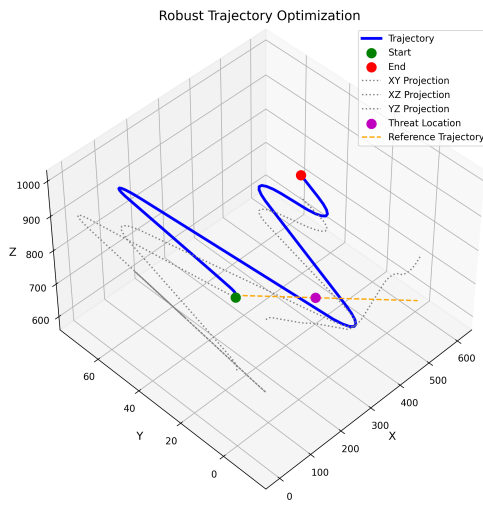


Fig. 6: Optimal Trajectory for the UAV using Robust Optimal Control

The g_{robust} constraint is met with a value of 0.99, indicating that the UAV has a high likelihood of evasion. The robust trajectory optimization algorithm is hence successful in ensuring that the UAV can evade the interceptor with a high likelihood by using Stochasticity as a means to account for the uncertainties in the environment.

IV. CONCLUSION

This project aimed to develop a robust trajectory optimization algorithm for UAVs using statistical inference and estimation techniques. The algorithm was implemented in two parts: first, learning the states and parameters of the interceptor using the Extended Kalman Filter and using the learned states and parameters for forecasting the future position of the interceptor. The second part was to optimize the trajectory of the UAV to avoid interception while accounting for the uncertainties in the environment. The optimization problem was formulated as an Optimal Control Problem using the Multiple Shooting Method. A robustness constraint was added to the optimization problem to ensure that the UAV has a high likelihood of evasion. The algorithm was tested on a simulated environment and the performance was evaluated based on the likelihood of